Uproszczony symulator blockchainu PoS

Vladyslav Rosol i Olesia Voronina

Grupa 22 INF-SP/B, Podstawy informatyki

C++17. FIFO i FEE w czasie dyskretnym.

1. Cel i zasady modelu

Projekt służy do pokazania wybranego fragmentu działania blockchainu Proof of Stake (PoS): oczekiwania transakcji na obsługę oraz wyboru walidatora z uwzględnieniem stake, czyli stawki. Porównujemy FIFO i FEE, mierząc oczekiwanie, dotrzymanie terminów i wykorzystanie walidatorów.

W sieci PoS, np. Ethereum, walidatorzy zabezpieczają środki, proponują bloki i sprawdzają bloki innych uczestników [1]. Nasz model przenosi pomysł wyboru według stawki na pojedyncze transakcje. Kolejka jest uproszczonym odpowiednikiem puli oczekujących transakcji. Fee oznacza opłatę określającą priorytet w strategii FEE.

Wybór transakcji i walidatora to dwie decyzje. FIFO wybiera wcześniejsze nadejście, a przy remisie mniejsze ID. FEE wybiera wyższe fee, następnie wcześniejsze nadejście i mniejsze ID. Walidatora losujemy spośród wolnych: jego szansa to stake / suma stake wolnych walidatorów. Przy stawkach 10 i 30 szanse wynoszą 25% i 75%, gdy obaj są wolni. Stake nie skraca czasu pracy.

Transakcja ma ID, nadejście, czas wykonania, fee i deadline. Walidator ma ID, dodatni stake i stan pracy. Wykonuje jedną transakcję naraz; wszyscy pracują jednakowo szybko. Pracy nie przerywamy, a spóźnione zadania również kończymy. Czas to całkowite kroki 0, 1, 2 itd., bez czekania rzeczywistej sekundy.

Zakres uproszczenia

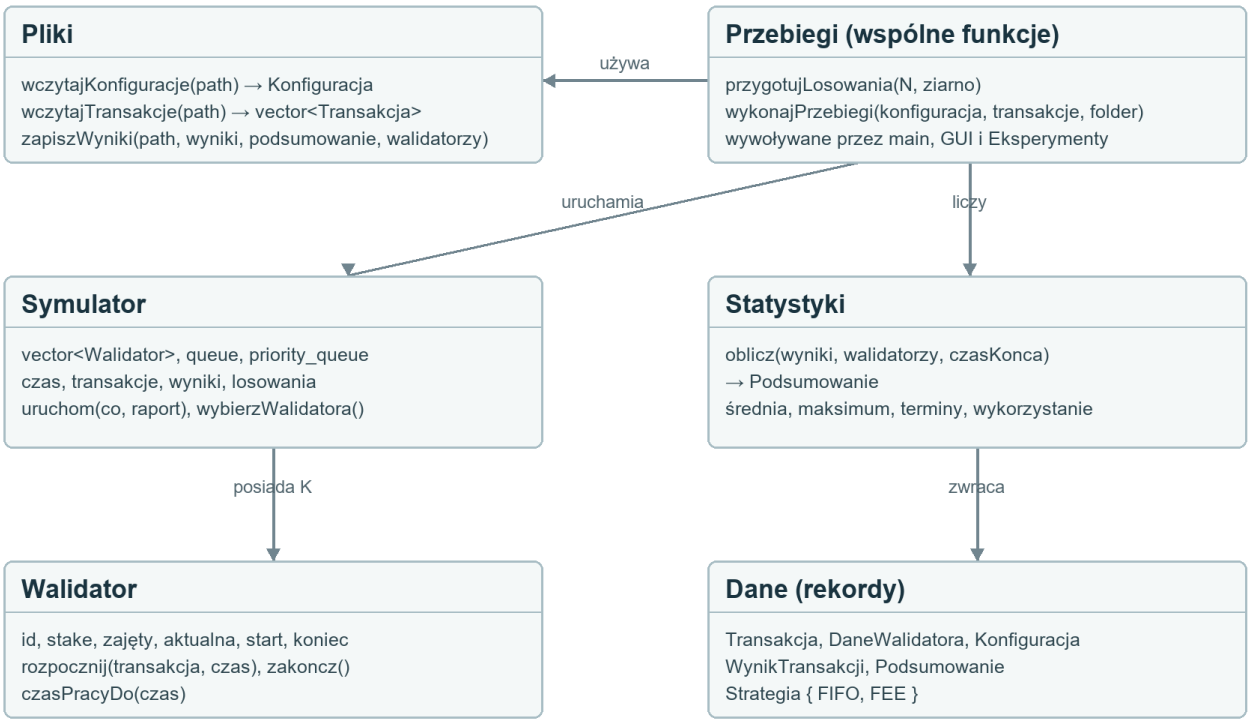
Program nie tworzy bloków ani ich łańcucha, nie sprawdza podpisów, sald czy poprawności przelewów. Pomijamy komunikację sieciową, uzgadnianie wspólnego łańcucha (konsensus), głosowania i ostateczne zatwierdzanie bloków. Nie ma nagród, kar, awarii ani zmian stake. Zakończenie transakcji oznacza upływ zadanego czasu obsługi, a nie potwierdzenie przez sieć. Deadline i równoległe stanowiska są założeniami zadania o obsłudze zgłoszeń. Wyniki opisują ten model kolejki, nie wydajność rzeczywistego blockchainu.

Przebieg jednego kroku

Dodajemy transakcje, które właśnie nadeszły. Dopóki są zadania i wolni wykonawcy, losujemy wykonawcę i pobieramy zadanie z wybranej kolejki. Jeśli została praca lub przyszłe nadejścia, zwiększamy czas o jeden, zapisujemy zakończenia i zwalniamy wykonawców. Potem wracamy do nadejść.

Transakcja rozpoczęta w 0 i trwająca 3 kroki kończy się w 3. W tym samym kroku wykonawca może rozpocząć kolejne zadanie. Pełny diagram działania znajduje się w diagramy.pdf.

2. Klasy i struktury danych



Klasy rozdzielają stan wykonawcy, symulację, pliki i pomiary. Typy wspólne są rekordami; pamięcią zarządzają kontenery STL.

Element	Odpowiedzialność
Walidator	Rozpoczęcie i zakończenie jednego zadania; czas pracy do chwili pomiaru.
Symulator	Czas, kolejki, losowania i zbieranie zakończeń. Posiada K obiektów Walidator.
Pliki	Odczyt, sprawdzenie pól, sortowanie wejścia i zapis wyników.
Statystyki	Końcowa średnia, maksimum, terminowość i wykorzystanie każdego wykonawcy.
Przebiegi	Funkcje wspólne: lista losowań oraz oddzielne uruchomienie FIFO i FEE.
Dane.hpp	Rekordy transakcji, walidatora, konfiguracji, wyniku i podsumowania.

vector przechowuje dane, wykonawców, losowania i wyniki. queue obsługuje FIFO. priority_queue używa funktora PorownajFee. Przed symulacją sortujemy transakcje po nadejściu i ID, a wykonawców po ID.

main.cpp obsługuje konsolę, Gui.cpp okno Windows, a Eksperymenty.cpp dane badawcze. Każdy korzysta z wykonajPrzebiegi. WidokWynikow.hpp sortuje indeksy tabeli bez zmiany oryginalnego wyniku. Sprawdzony diagram klas z FigJam jest w diagramy.pdf.

3. Dane i sposób przeprowadzenia badań

W każdym scenariuszu użyliśmy 6000 transakcji i 300 walidatorów. Dane są syntetyczne. Generator w C++ używa mt19937 z ziarnem 20261102. Dla danej transakcji zachowujemy ten sam czas wykonania i fee pomiędzy scenariuszami. Zmieniamy nadejście i przesuwamy deadline, zachowując ten sam zapas czasu.

Parametr	Wartość
ID	Transakcje 1 do 6000, walidatorzy 1 do 300.
Stake	10 + (ID × 37) mod 991, stały przez cały przebieg.
Czas wykonania	8 + generator() mod 25, czyli 8 do 32 kroków. Wartości powtarzają się.
Fee	1 + generator() mod 100, czyli 1 do 100.
Deadline	nadejście + czas wykonania + 10 + floor((100-fee)/2).

floor oznacza część całkowitą wyniku. Większe fee ma mniejszy zapas deadline: od 10 do 59 kroków. Jest to założenie tych danych, a nie cecha wszystkich systemów PoS.

Scenariusz	Nadejścia	Warunki pracy
Małe obciążenie	6 na krok, kroki 0 do 999.	Zapas zasobów; w danych nie ma oczekiwania.
Przeciążenie	100 na krok, kroki 0 do 59.	Kolejka rośnie, bo napływ przekracza wydajność.
Nagły napływ	Wszystkie 6000 w kroku 0.	Jednorazowa długa kolejka.

Przy czasie wykonania około 20 kroków 300 wykonawców obsługuje w przybliżeniu 15 zadań na krok. Sześć nowych zadań mieści się poniżej tej wydajności, a sto ją przekracza. Domyślny zgloszenia.txt jest kopią przeciazenie.txt.

Badania obejmują 18 przebiegów: trzy scenariusze, FIFO i FEE oraz ziarna 11, 22 i 33. Obie strategie zaczynają od nowego stanu i pierwszej pozycji tej samej listy N losowań.

Wyniki zachowaliśmy w eksperymenty/scenariusz/ziarno. porownanie.csv zawiera miary całych przebiegów, wykorzystanie.csv dane każdego walidatora, a grupy_fee.csv podział na niskie i wysokie opłaty.

4. Miary

Oczekiwanie = start - nadejście. Opóźnienie = max(0, koniec - deadline). Zadanie kończące się dokładnie w deadline jest terminowe. Dla pustego wejścia wszystkie miary wynoszą zero.

Średnią i maksimum oczekiwania oraz liczbę terminowych i spóźnionych liczymy dla zakończonych transakcji. Wykorzystanie walidatora to 100% × jego czas pracy / czas od 0 do chwili pomiaru. Średnie wykorzystanie obejmuje wszystkich wykonawców, także tych bez zadań. Raporty co 100 są narastające, a wykorzystanie w trakcie obejmuje już wykonaną część zadań w toku. Ostatni raport obejmuje również niepełną setkę.

5. Wyniki i porównanie

Czasy podajemy w krokach, wykorzystanie jako średnią dla 300 walidatorów. Dla ziaren 11, 22 i 33 zbiorcze miary są identyczne, więc każdy wiersz przedstawia trzy przebiegi. Wszystkie 18 wyników pozostaje w porownanie.csv.

Scenariusz	Strategia	Ziarna	Śr. czekanie	Max	Terminowe	Spóźnione	Wykorzystanie %
Małe obciążenie	FIFO	11/22/33	0.000	0	6000	0	39.06
Małe obciążenie	FEE	11/22/33	0.000	0	6000	0	39.06
Przeciążenie	FIFO	11/22/33	161.728	333	787	5213	94.98
Przeciążenie	FEE	11/22/33	162.054	384	1014	4986	94.98
Nagły napływ	FIFO	11/22/33	190.227	391	690	5310	95.20
Nagły napływ	FEE	11/22/33	190.564	391	361	5639	95.43

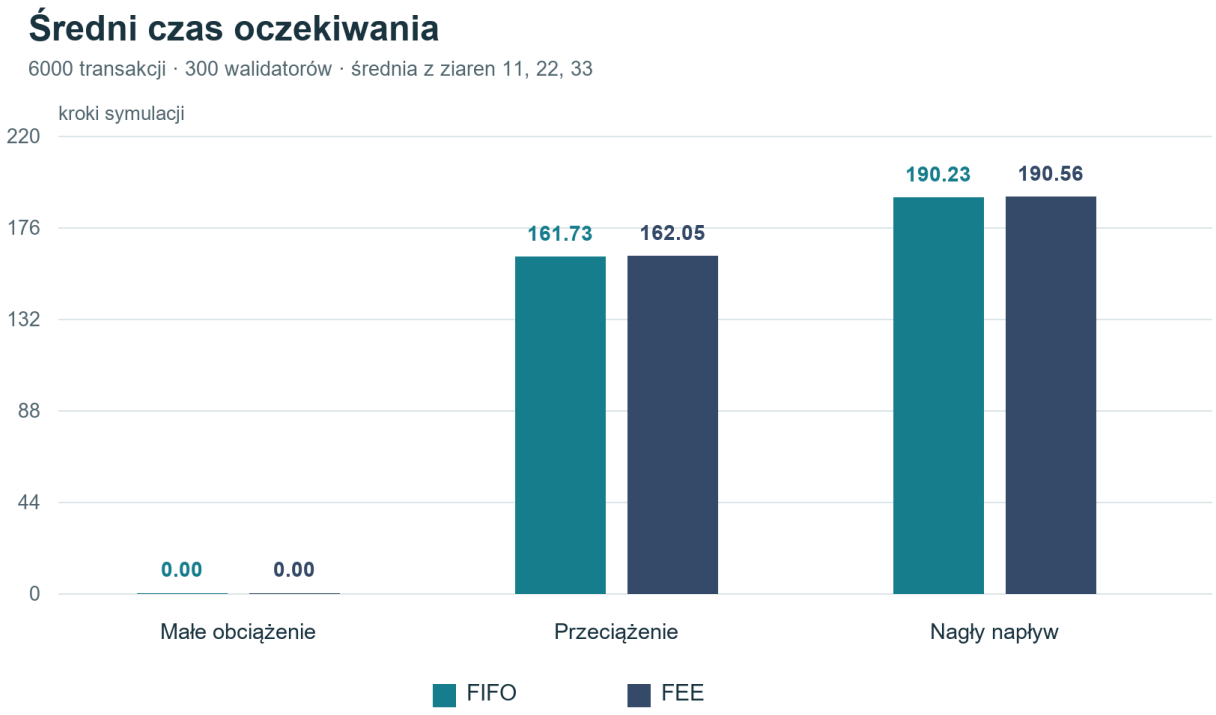
Zmiana ziarna zmienia ID przydzielonych walidatorów, ale przy ich jednakowej szybkości zachowuje harmonogram transakcji. Przy małym obciążeniu w FIFO największe wykorzystanie pojedynczego walidatora wyniosło 69,16%, 70,51% i 66,73%; średnia grupy pozostała 39,06%. Powtarzalność przydziałów dotyczy tego samego środowiska: uniform_real_distribution może inaczej przekształcać losowania w innej bibliotece standardowej.

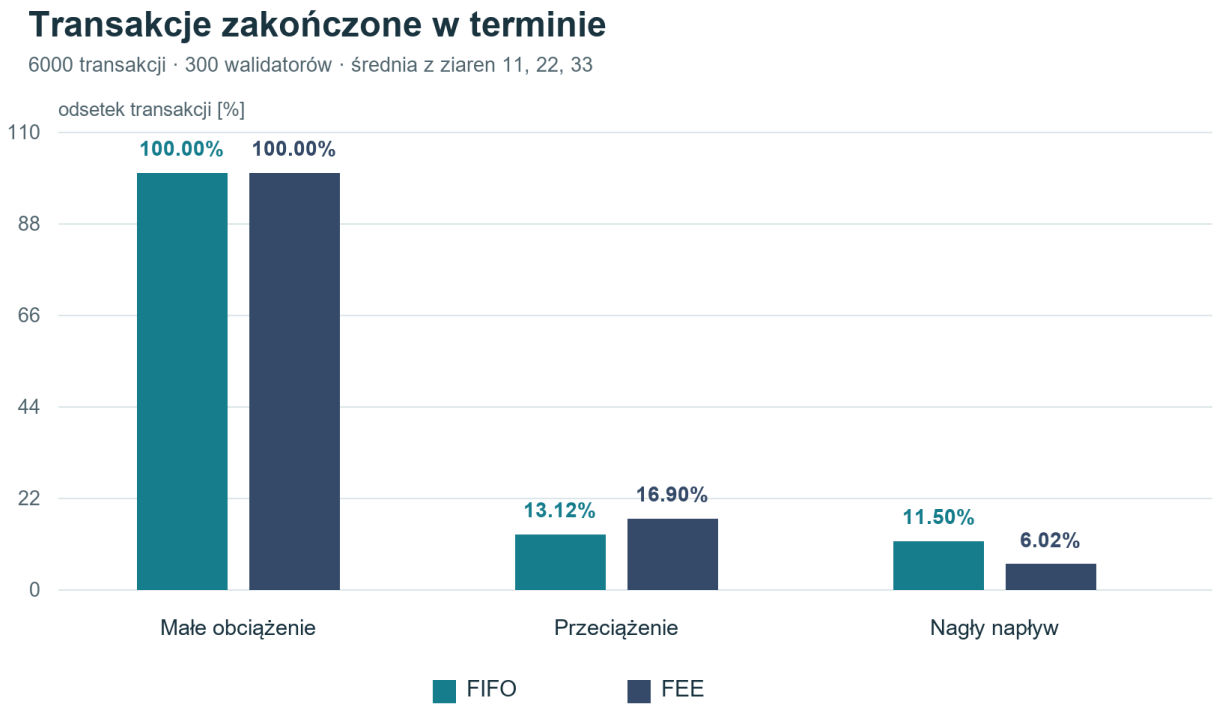
Scenariusz	Strategia	Grupa fee	Średnie oczekiwanie	Terminowe %
Przeciążenie	FIFO	1 do 25	162,51	18,55
Przeciążenie	FIFO	75 do 100	161,68	7,38
Przeciążenie	FEE	1 do 25	300,30	5,32
Przeciążenie	FEE	75 do 100	28,00	50,45
Nagły napływ	FIFO	1 do 25	191,14	15,82
Nagły napływ	FIFO	75 do 100	190,16	6,81
Nagły napływ	FEE	1 do 25	344,93	0,00
Nagły napływ	FEE	75 do 100	42,47	22,96

Grupy fee 1 do 25 i 75 do 100 liczą odpowiednio 1391 i 1572 transakcje. Tabela dotyczy ziarna 11; pozostałe dają te same miary grup. Środkowy zakres fee pozostaje uwzględniony w wynikach całego przebiegu.

FEE uprzywilejowuje wysokie opłaty kosztem dłuższego oczekiwania niskich. W przeciążeniu poprawia terminowość z 13,12% do 16,90%, ale zwiększa maksimum oczekiwania z 333 do 384. Przy nagłym napływie obniża terminowość z 11,50% do 6,02%: wysoka opłata ma tu krótszy zapas deadline, a FEE nie wybiera najbliższego terminu. Przy małym obciążeniu obie strategie dają 100% terminowości i brak oczekiwania. Żadna nie wygrywa we wszystkich miarach.

6. Wykresy





Wykresy pokazują średnie z trzech ziaren. Dla przedstawionych miar wartości każdego ziarna są identyczne. Są to pomiary przebiegu w krokach, a nie pomiary czasu działania komputera w sekundach.

7. Złożoność i ograniczenia

N oznacza liczbę transakcji, K liczbę walidatorów, a T liczbę zwiększeń czasu. B to odstęp raportów, domyślnie 100. Liczba raportów R nie przekracza floor(N/B)+1.

Operacja	Koszt czasu
FIFO push, pop, front	O(1).
FEE push i pop; top	O(log(N+1)); top O(1).
Sprawdzenie ID i sortowanie wejścia	O(N log(N+1) + K log(K+1)), przy stałej liczbie pól.
Skany wykonawców w czasie	O((T+1)K), także dla pustego przebiegu.
Wybór wykonawcy dla N przydziałów	O(NK).
Końcowe wykorzystanie	O(NK): K wykonawców i N wyników.
Raporty narastające	O(RK).
Całość silnika	O(N log(N+1) + K log(K+1) + (T+1)K + NK + RK).
Pamięć silnika	O(N+K).

Ponieważ R jest O(N+1), raporty nie zmieniają ogólnej granicy. Dwa przebiegi mnożą koszt przez stałą. GUI przechowuje indeksy wierszy O(N). Sortowanie widoku ma O(N log(N+1)) porównań przy dostępnym dodatkowym buforze; bez niego stable_sort może wymagać O(N log²(N+1)). Skany wykonawców są w Symulator.cpp, a końcowe statystyki w Statystyki.cpp.

Długie przerwy w nadejściach zwiększają T, bo pętla sprawdza każdy krok. Przy ciągłym napływie wysokich opłat FEE może głodzić niskie opłaty. Nasze pliki są skończone, więc wszystkie zadania kończymy; mierzymy długie oczekiwanie.

Dane syntetyczne i trzy ziarna nie opisują wszystkich warunków. Model pozwala badać kolejki i podział pracy według stake; bezpieczeństwo i konsensus PoS pozostają poza zakresem.

8. Sprawdzenie poprawności

Mały przykład daje FIFO 1,2,3 i FEE 2,3,1, średnie oczekiwanie 2, maksimum 4 i wykorzystanie 100%. Testy obejmują remisy, granice losowania stake, pomijanie zajętych, puste i błędne wejście, dokładny deadline, przerwy w pracy i raporty 100/200/205. Sprawdzamy także równoczesne zakończenia i pracę w toku.

Test C++ odtwarza 18 przebiegów, porównuje miary z CSV i sprawdza 108000 rekordów: ID, czasy, brak nakładania pracy i wykorzystanie. Przy weryfikacji wydania sprawdzono też powtarzalność plików oraz GUI: błąd zapisu, zmianę wejścia, sortowanie i zamknięcie podczas obliczeń.

9. Uruchomienie i odtworzenie

Na Windows uruchamiamy symulator_gui.exe lub symulator.exe. GUI ma wybór plików, zakładki FIFO/FEE i tabelę wyników. Kliknięcie nagłówka sortuje widok, a przycisk przywraca kolejność zakończenia. Obliczenia działają w osobnym wątku.

Konsola czyta konfiguracja.txt i zgloszenia.txt. Parametry --konfiguracja, --zgloszenia i --wyniki wybierają ścieżki; --ziarno ustawia losowania, --co odstęp raportów, --cicho wyłącza komunikaty, a --pomoc pokazuje składnię. GUI zapisuje w folderze wyniki obok pliku transakcji; konsola w bieżącym folderze lub wskazanym przez --wyniki. Kolejny przebieg nadpisuje wyniki FIFO/FEE.

Kompilacja: .\build.cmd. Testy C++: .\testy.cmd. Oba działania: .\build.cmd --tests. Wymagany jest GCC/MinGW z C++17 i windres w PATH. Gotowe EXE działają bez tych narzędzi. GUI korzysta z Win32.

Eksperymenty.exe odtwarza i nadpisuje konfigurację, duże zestawy, CSV i wyniki badań. Mały przykład jest w przyklady/. Sprawozdanie i diagramy są dla prowadzącego; osobny przygotowanie_do_obrony.pdf zawiera plan pokazu do 5 minut, pytania z odpowiedziami i ćwiczenia.

10. Pytania o PoS i zakres projektu

Co tutaj przedstawia Proof of Stake? Losowanie wolnego walidatora z szansą proporcjonalną do stake. Większa stawka daje większą szansę przydziału, a nie szybszą obsługę.

Czy FEE jest algorytmem konsensusu? FEE ustala kolejność transakcji. Konsensus, czyli uzgodnienie wspólnego stanu blockchainu przez uczestników, jest pominięty.

Co oznacza zakończona transakcja? Wykonanie zadania przez walidatora po zadanej liczbie kroków. Model nie potwierdza przelewu ani dodania bloku do łańcucha.

Co można dalej rozbudować? Tworzenie bloków z wielu transakcji, powiązania między blokami, głosowania walidatorów oraz nagrody i kary. W obecnej wersji skupiamy się na wyborze wykonawców i porównaniu kolejek.

Wymagania i źródła

Kod i model realizują K_W05 i K_U15, porównanie badań K_U05, dobór algorytmów K_U08, a analiza złożoności K_U09. Sprawozdanie i wspólny pokaz wspierają K_K09 oraz K_K10; wiedzę autorów ocenia prowadzący podczas obrony.

Wyniki pochodzą z kodu C++, danych wejściowych, trzech CSV i pełnych przebiegów. Diagramy działania i klas w diagramy.pdf pochodzą z naszej tablicy FigJam: https://www.figma.com/board/ITzrf0DeaTyMBG1xbEF7J7/Untitled .

[1] Ethereum.org, Proof-of-stake (PoS): https://ethereum.org/en/developers/docs/consensus-mechanisms/pos/ . Źródło objaśnia rzeczywisty mechanizm; nasz model obejmuje opisane w rozdziale 1 uproszczenia.